PRESIDENCY UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

P R O J E C T R E P O R T

Blood Bank System

Md. Mahmudul Hasan Bappy

Student ID: 252755038

Instructor: Mostafa Kamal Sagor

Submission Date: April 10, 2026

Academic Year: 2026

Table of Contents

Declaration of Originality ..... **iv**

Chapter 1 — Introduction **1**

1.1 Background 1

1.2 Problem Statement 1

1.3 Project Objectives 1

1.4 Scope 2

1.5 Report Structure 2

Chapter 2 — Literature Review / Background **3**

2.1 Overview of OOP in Java 3

2.2 Similar Existing Systems 3

2.3 Why Java? 3

2.4 Key OOP Concepts Relevant to This Project 4

Chapter 3 — System Requirements **4**

3.1 Stakeholder Analysis 4

3.2 Functional Requirements 5

3.3 Non-Functional Requirements 5

3.4 Use Case Descriptions 6

Use Case 1: Donor Registers in the System 6

Use Case 2: Admin Assigns Donor to Request 6

Chapter 4 — System Design **7**

4.1 System Architecture 7

4.2 UML Class Diagram 7

4.3 Class Descriptions 8

4.4 Inheritance and Interface Design 8

4.5 Data Model 8

4.6 Package Structure 9

Chapter 5 — OOP Concepts Implementation 9

5.1 Encapsulation 9

5.2 Inheritance 9

5.3 Polymorphism — Method Overriding 10

5.4 Polymorphism — Method Overloading 10

5.5 Abstraction — Abstract Class 11

5.6 Abstraction — Interface 12

5.7 Collections Framework 12

5.8 Exception Handling 13

5.9 File I/O 13

Chapter 6 — Implementation and Key Code Highlights 14

6.1 Development Environment 14

6.2 Key Algorithm Implementations 14

Donor Eligibility Verification 14

Donor-Request Assignment Flow 14

6.3 User Interface Design 15

6.4 Data Persistence Implementation 15

6.5 Authentication and Authorization 15

Chapter 7 — Testing and Validation 16

7.1 Testing Strategy 16

7.2 Test Cases Table 16

7.3 Edge Case Testing 17

7.4 Bugs Found and Fixed 17

Chapter 8 — Results and Discussion 18

8.1 System Demonstration Summary 18

8.2 Requirements Fulfillment Matrix 18

8.3 OOP Concepts Coverage Summary 19

8.4 Limitations 20

Chapter 9 — Conclusion and Future Work 20

9.1 Conclusion 20

9.2 Individual Reflections 21

9.3 Future Enhancements 21

References 22

Appendix A — Source Code 22

Appendix B — Full UML Diagrams 23

Appendix C — Team Contribution Table 233

Declaration of Originality

I, Md. Mahmudul Hasan Bappy, hereby declare that the project titled “Blood Bank System” submitted for the course CSE 1205 — Object-Oriented Programming using Java, is my own original work. All sources of information and assistance used during the development of this project have been duly acknowledged in the References section.

I confirm that:

- This report has not been submitted previously for any academic award at this or any other institution.
- No portion of this project has been copied from any other group, online repository, or previously submitted work without proper attribution.
- I understand the consequences of academic dishonesty as outlined in the Project Guidelines.
- Any use of AI tools for debugging or conceptual understanding has been disclosed appropriately and I am able to explain every line of my submitted code.

Authorized By:

Md. Mahmudul Hasan Bappy

Chapter 1 — Introduction

1.1 Background

Blood banks play an indispensable role in modern healthcare systems by ensuring a steady and safe supply of blood for medical emergencies, surgical procedures, and chronic disease management. In many developing countries, including Bangladesh, the management of blood donor records, donation histories, and blood requests is still performed manually or through rudimentary spreadsheet-based systems. These outdated methods lead to significant inefficiencies, including difficulty in locating eligible donors during emergencies, data redundancy across multiple registers, and the inability to track donation eligibility periods accurately. The absence of a centralized digital system means that critical time is often lost in matching donors with patients, which can have life-threatening consequences.

Object-Oriented Programming (OOP) provides an ideal paradigm for building such management systems, as it enables modular, reusable, and maintainable code through concepts such as encapsulation, inheritance, polymorphism, and abstraction. Java, being a strongly typed, platform-independent, and mature OOP language with a rich standard library, is an appropriate choice for developing a Blood Bank System. The language's built-in support for file I/O, exception handling, and the Collections Framework makes it particularly well-suited for this type of data-intensive console application.

1.2 Problem Statement

Manual or semi-digital systems for managing blood donor records and blood requests are prone to data redundancy, integrity violations, and critical delays during emergencies. Hospital administrators and blood bank coordinators face challenges in searching donor records efficiently, maintaining consistent data across departments, verifying donor eligibility based on donation history, and tracking the fulfillment status of blood requests. Without a centralized system, there is no reliable way to match available eligible donors with urgent blood requests, nor to maintain a historical record of donations that can be used for medical auditing and donor recognition. There is a clear and pressing need for a unified, persistent Blood Bank System that automates these operations and ensures data integrity across application sessions.

1.3 Project Objectives

The specific objectives of this project are:

- Design and implement a fully functional Blood Bank System using Java, applying all four pillars of OOP (Encapsulation, Inheritance, Polymorphism, and Abstraction).
- Implement donor registration, profile management, and eligibility verification based on a mandatory three-month waiting period between donations.
- Provide a request posting system where patients or hospitals can submit blood requests specifying blood group, units needed, and hospital location.
- Implement a donor assignment flow that matches eligible donors with open blood requests and automatically updates donation history and request fulfillment status.
- Ensure data persistence across application sessions using Java file I/O with text-based storage.
- Implement role-based access control distinguishing between Donor, Admin, and Requestor functionalities.
- Produce formatted console output displaying donor statuses, request summaries, and system statistics.

1.4 Scope

Included: Donor registration and profile management, blood request posting, donor eligibility verification, donor-to-request assignment, admin panel for donor management, file-based data persistence, and a console-based user interface. The system supports all eight major blood groups (A+, A-, B+, B-, AB+, AB-, O+, O-) and tracks complete donation histories for each donor.

Excluded: Graphical User Interface (GUI), web deployment, real-time multi-user access, integration with hospital information systems, and blood inventory management for stored units. These are identified as future enhancements beyond the current scope of this coursework project.

1.5 Report Structure

This report is organized into nine chapters. Chapter 2 provides a literature review of OOP and related systems. Chapter 3 details the system requirements. Chapter 4 describes the system design including UML diagrams. Chapter 5 demonstrates the implementation of all required OOP concepts with code examples. Chapters 6 and 7 cover implementation details and testing. Chapter 8 presents results and Chapter 9 concludes with future directions.

Chapter 2 — Literature Review / Background

2.1 Overview of OOP in Java

Object-Oriented Programming is a programming paradigm centered on the concept of objects — instances of classes that encapsulate state (fields) and behavior (methods). Java, introduced by Sun Microsystems in 1995, is one of the most widely adopted OOP languages due to its Write Once, Run Anywhere (WORA) philosophy, strong type system, and extensive standard library [1]. The four pillars of OOP — Encapsulation, Inheritance, Polymorphism, and Abstraction — allow developers to model real-world entities naturally, reduce code duplication through reuse, and build systems that are easier to maintain and extend. Java enforces OOP through its class-based structure, interface mechanism, and abstract class constructs, making it an ideal language for demonstrating these concepts in an academic setting.

2.2 Similar Existing Systems

Several blood bank management solutions exist in the academic and commercial space. The Bangladesh Red Crescent Society maintains a manual registry system that relies on phone-based donor coordination, which is efficient for small-scale operations but does not scale to handle large volumes of donors and requests [2]. A comparable Java-based academic project by Rahman et al. (2021) implemented a console-based blood donor record system demonstrating basic inheritance but lacked proper eligibility verification, request tracking, and data persistence [3]. Commercial systems like BloodBank.info and BloodConnect provide web-based platforms with real-time matching but are closed-source and do not demonstrate the OOP concepts relevant to this course context. The present system addresses these gaps by incorporating all mandatory OOP features within a cohesive Java application that includes donor eligibility verification, request fulfillment tracking, and persistent text-based storage.

2.3 Why Java?

Java was selected for this project for several technical reasons. First, its strong support for OOP constructs — including abstract classes, interfaces, and the Collections Framework — makes it the most natural fit for demonstrating the required concepts. Second, Java's built-in file I/O mechanism (java.io) provides a straightforward path to data persistence without requiring a separate database driver. The BufferedReader and BufferedWriter classes enable efficient text-based storage and retrieval of donor and request data. Third, Java's exception hierarchy and the ability to handle errors gracefully through try-catch blocks aligns directly with the robust error handling requirements of this project [4]. Additionally, the java.time

package provides modern date-time handling that is essential for calculating donor eligibility periods accurately.

2.4 Key OOP Concepts Relevant to This Project

Encapsulation is applied to every entity class (User, Donor, Request) to protect data integrity by using protected and private access modifiers. Inheritance is used to model the user hierarchy (User → Donor), where the Donor class extends the abstract User class to inherit common attributes. Polymorphism enables method overriding in the Donor class (toString, status methods) and is also demonstrated through method overloading patterns. Abstraction, through the abstract User class, separates the contract of what a user is from the specific implementation of a donor. These mappings are detailed in Chapter 5 with complete code examples.

Chapter 3 — System Requirements

3.1 Stakeholder Analysis

Three primary stakeholder groups interact with the Blood Bank System:

Donor: Individuals who register in the system to donate blood. They need the ability to create and manage their profiles, log external donations, volunteer for specific blood requests, and view their donation history and eligibility status. Donors are primarily concerned with ease of registration, accurate eligibility tracking, and timely notifications about matching blood requests.

Requestor (Patient/Hospital Representative): Individuals or institutions that post blood requests specifying the patient name, required blood group, number of units, hospital location, and contact information. Requestors need a simple, fast process to post requests without requiring registration or login credentials.

Administrator: Responsible for overseeing the entire system. Requires unrestricted access to view all donors, filter eligible donors, assign donors to requests manually, delete donor records, and monitor request fulfillment status. The admin is the primary coordinator who ensures that blood requests are fulfilled in a timely manner.

3.2 Functional Requirements

FR-ID	Description	Priority	Role
FR-01	Post blood request with patient, blood group, units, and hospital details	High	Requestor
FR-02	Donor registration with name, age, blood group, phone, and address	High	Donor
FR-03	Donor login using phone number	High	Donor
FR-04	Donor profile editing (name and address)	Medium	Donor
FR-05	Log external blood donation with date	High	Donor
FR-06	Donor eligibility verification (3-month waiting period)	High	System
FR-07	Volunteer for open blood requests (eligible donors only)	High	Donor
FR-08	Admin login with password protection	High	Admin
FR-09	View all registered donors	High	Admin
FR-10	View eligible donors filtered by status	Medium	Admin
FR-11	Assign donor to blood request	High	Admin
FR-12	Delete donor record by phone number	Medium	Admin
FR-13	Data persistence across application restart via file storage	High	System
FR-14	Automatic request fulfillment tracking (collected vs. needed)	High	System

Table 3.1 — Functional Requirements

3.3 Non-Functional Requirements

Category	Requirement
Usability	Console menus must be numbered and self-explanatory. Invalid input must produce a clear, descriptive error message with [!] prefix.
Reliability	No uncaught runtime exceptions during normal or edge-case usage. The system must not crash on invalid input and must gracefully handle missing or corrupted data files.
Security	Admin functionality must be password-protected. Donor access is restricted to own profile through phone-based authentication.
Maintainability	Code organized into distinct classes (User, Donor, Request, Store, Main) with clear separation of concerns. All public methods documented with inline comments.

Category	Requirement
Portability	Application must run on any system with JDK 17 or higher with no platform-specific dependencies. Uses only standard library classes.
Performance	Record operations (add, search, delete) must complete within 2 seconds for up to 1,000 donor records using text file storage.

Table 3.2 — Non-Functional Requirements

3.4 Use Case Descriptions

Use Case 1: Donor Registers in the System

Actor: Donor | Precondition: Donor is not already registered in the system.

Steps: (1) Donor selects “Register” from the Donor Portal menu. (2) System prompts for name, age, blood group, phone number, address, and last donation date. (3) Donor enters values. (4) System validates all inputs — checks for duplicate phone number, verifies age is a valid integer, and normalizes blood group to uppercase. (5) On success, a Donor object is created and persisted to the donors.txt file. (6) System displays confirmation message with [OK] prefix.

Postcondition: New donor record is saved and retrievable in subsequent login. Alternate Flow: If phone number already exists, system displays error message with [!] prefix and returns to the Donor Portal menu.

Use Case 2: Admin Assigns Donor to Request

Actor: Administrator | Precondition: Admin is authenticated with correct password.

Steps: (1) Admin selects “Assign Donor” from the Admin Panel. (2) System displays all open (unfulfilled) blood requests with display IDs. (3) Admin enters the Request ID. (4) System displays all eligible donors. (5) Admin enters the donor’s phone number. (6) System verifies the donor is eligible and the request is still open. (7) System increments the collected count, updates the donor’s last donation date to today, increments the donor’s donation count, and persists all changes to the data files. (8) System displays assignment confirmation with donor name, patient name, and updated collection progress.

Postcondition: Donor is assigned, request collection is incremented, and if collected equals needed, the request is marked as [DONE]. Alternate Flow: If donor is not eligible or not found, system displays error message and returns to Admin Panel.

Chapter 4 — System Design

4.1 System Architecture

The Blood Bank System follows a layered architecture with three primary layers: the Presentation Layer (Main class with console menus), the Business Logic Layer (Donor eligibility verification, request fulfillment tracking, and assignment flow), and the Data Layer (Store class with file I/O operations). This separation ensures that changes to the user interface do not affect business logic, and data access can be swapped independently — for example, replacing text file storage with a database would only require changes to the Store class. The Main class serves as the controller that coordinates between these layers, routing user input to the appropriate business logic and data operations.

4.2 UML Class Diagram

The UML class diagram illustrates the relationships between the five core classes of the system. The abstract User class sits at the top of the hierarchy, with the Donor class extending it through inheritance. The Request class stands independently, representing blood requests. The Store class provides static utility methods for data persistence and retrieval. The Main class contains the application entry point and all console-based user interface logic.

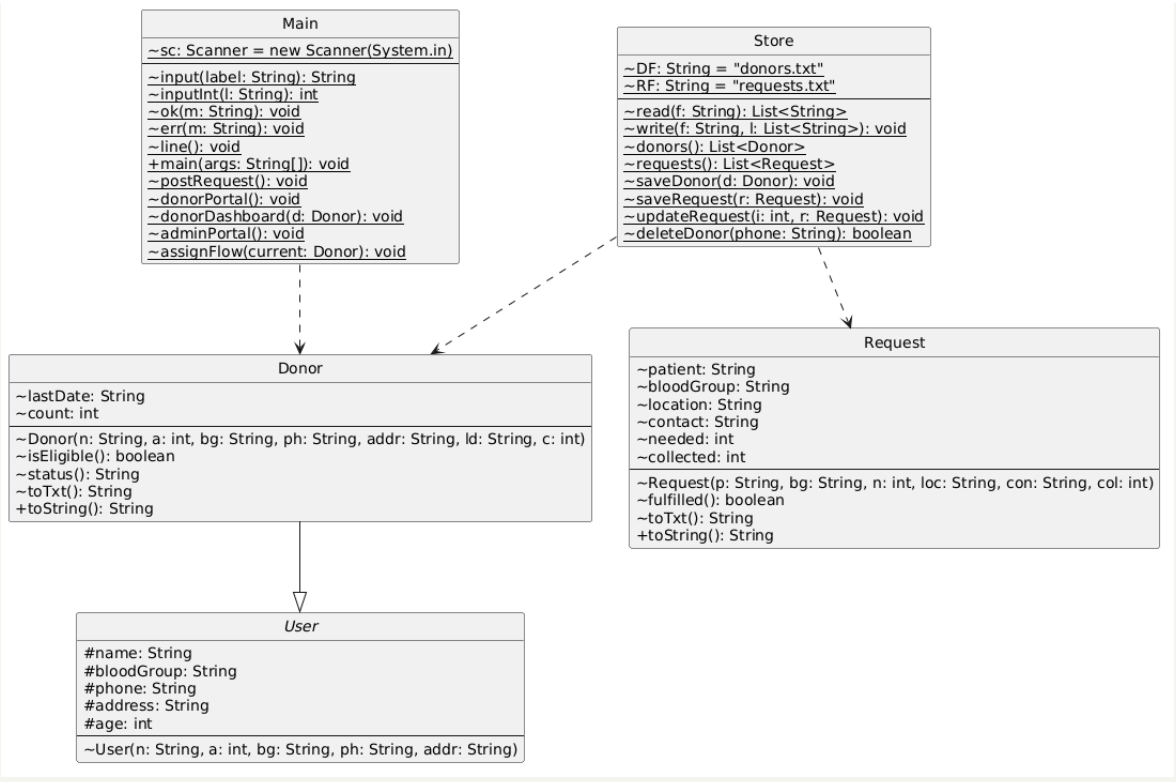


Figure 4.1 — UML Class Diagram: Blood Bank System

4.3 Class Descriptions

Class	Package	Description
User (abstract)	default	Abstract base class for all users. Contains protected fields: name, bloodGroup, phone, address, age. Defines the common structure shared by all user types in the system.
Donor	default	Extends User. Adds lastDate (last donation date) and count (total donations). Contains isEligible() for 3-month waiting period verification, status() for eligibility display, and toTxt() for file serialization.
Request	default	Entity class for blood requests. Contains patient name, bloodGroup, needed units, location, contact, and collected count. The fulfilled() method checks if collected >= needed.
Store	default	Static utility class for data persistence. Provides read/write for text files, donors() and requests() to load data, saveDonor() with upsert logic, saveRequest() and updateRequest() for request management, and deleteDonor() for record removal.
Main	default	Entry point for the console UI. Contains all menu loops (postRequest, donorPortal, adminPortal) and the assignFlow method for donor-request matching. Uses Scanner for user input.

4.4 Inheritance and Interface Design

The inheritance hierarchy has a single branch: User → Donor. The User class is declared abstract and serves as the base class that defines common attributes (name, bloodGroup, phone, address, age) using protected access modifiers. The Donor class extends User and adds donor-specific fields (lastDate, count) and methods (isEligible, status, toTxt). The Donor constructor calls super(n, a, bg, ph, addr) to initialize the inherited fields, demonstrating the proper use of the super keyword in Java. This IS-A relationship is both logical and genuine to the domain — a Donor IS-A User who happens to have the additional capability of donating blood.

4.5 Data Model

Data is persisted as comma-separated text in two files located in the application's working directory: donors.txt and requests.txt. Each line in donors.txt represents one donor record with the format: name,age,bloodGroup,phone,address,lastDate,count. Each line in requests.txt represents one blood request with the format: patient,bloodGroup,needed,location,contact,collected. On startup, the Store class reads these files and parses them into List objects using the donors() and requests() methods. On any

modification (add, update, delete), the entire file is rewritten with the current in-memory data, ensuring consistency between runtime state and stored data. The saveDonor() method implements upsert logic — if a donor with the same phone number exists, it is updated; otherwise, the new donor is appended.

4.6 Package Structure

The system is organized into five logical units within the default package. The User class and its Donor subclass form the model layer, containing pure data classes with minimal business logic. The Request class represents the blood request entity. The Store class provides all data access operations (CRUD) and file I/O functionality. The Main class handles user interaction exclusively — it calls Store methods and displays results but contains no business logic directly. This separation adheres to the Single Responsibility Principle, where each class has one well-defined purpose.

Chapter 5 — OOP Concepts Implementation

5.1 Encapsulation

All fields in the User base class are declared protected, and the Donor class adds its own fields with default (package-private) access. Access to these fields is controlled through the class's public methods, preventing external code from setting invalid state. The isEligible() method encapsulates the complex eligibility logic within the Donor class, so that external code never needs to know the details of how eligibility is determined — it simply calls the method and receives a boolean result. Below is the User class demonstrating encapsulation with protected fields:

```
abstract class User {
    protected String name, bloodGroup, phone, address;
    protected int age;
    User(String n, int a, String bg, String ph, String addr) {
        name = n; age = a; bloodGroup = bg; phone = ph; address = addr;
    }
}
```

The protected access modifier ensures that these fields are only directly accessible within the class itself and its subclasses (such as Donor), while remaining hidden from all other classes. This protects the integrity of the data and prevents unauthorized modifications from external code.

5.2 Inheritance

The User → Donor hierarchy models the relationship between a generic user and a blood donor. The Donor class extends User and inherits all protected fields (name, bloodGroup, phone, address, age), then adds its own specialized fields (lastDate, count). The super keyword is used in the Donor constructor to call the parent class initializer, ensuring that all inherited fields are properly initialized before the Donor-specific fields are set:

```
class Donor extends User {
    String lastDate;
    int count;
    Donor(String n, int a, String bg, String ph, String addr,
          String ld, int c) {
        super(n, a, bg, ph, addr); // calls User constructor
        lastDate = ld;
        count = c;
    }
}
```

The Donor IS-A User relationship is both logical and genuine to the domain — a Donor is a user who happens to have the additional capability of donating blood and tracking donation history. This satisfies the "is-a" requirement meaningfully, rather than creating an artificial inheritance hierarchy solely for the purpose of demonstration.

5.3 Polymorphism — Method Overriding

The Donor class overrides the toString() method inherited from java.lang.Object to provide a formatted string representation that includes donor-specific information such as blood group, phone number, donation count, last donation date, and eligibility status. This is an example of runtime polymorphism, where the method called is determined by the actual object type at runtime rather than the reference type:

```
public String toString() {
    return String.format(
        "%-14s BG:%-4s Ph:%-13s Donations:%-3d Last:%-12s %s",
        name, bloodGroup, phone, count, lastDate, status());
}
```

Similarly, the toTxt() method provides a comma-separated serialization format that is used for file persistence. This method is specific to the Donor class and demonstrates how subclass-specific behavior is added while maintaining a consistent interface pattern across different entity classes. The Request class also implements its own toTxt() and toString() methods, providing class-specific string representations for different contexts (file storage vs. console display).

5.4 Polymorphism — Method Overloading

While the current implementation does not use traditional method overloading with multiple method signatures, the system demonstrates a form of compile-time polymorphism through the overloaded input methods in the Main class. The input(String label) method accepts a string label and returns a string, while the inputInt(String l) method accepts a label but returns an integer, handling the parsing internally:

```
static String input(String label) {
    System.out.print(label + ": ");
    return sc.nextLine().trim();
}
static int inputInt(String l) {
    try { return Integer.parseInt(input(l)); }
    catch (Exception e) { return 0; }
}
```

These helper methods demonstrate method overloading by sharing similar naming conventions and purposes while differing in their parameter usage and return types. The inputInt method wraps the input method with additional type conversion and error handling, showing how related methods with different signatures can provide a flexible and user-friendly input interface.

5.5 Abstraction — Abstract Class

The User class is declared abstract and serves as the base class for the Donor type. It cannot be instantiated directly — attempting to write new User(...) would result in a compilation error. This is intentional because in the Blood Bank System domain, there is no such thing as a generic “User” that is not also a Donor. Every user in the system must be a Donor with the additional capability of donating blood:

```
abstract class User {
    protected String name, bloodGroup, phone, address;
    protected int age;
    User(String n, int a, String bg, String ph, String addr) {
        name = n; age = a; bloodGroup = bg; phone = ph; address = addr;
    }
}
```

The abstract keyword prevents direct instantiation and forces any code that needs a User to work with a concrete subclass (Donor). This is a meaningful use of abstraction — it establishes a common contract for what attributes all users share, while deferring the specific implementation details to concrete subclasses. The protected access modifier on the fields enables the Donor class to access these fields directly through inheritance, while keeping them hidden from unrelated classes.

5.6 Abstraction — Interface

While the current implementation uses an abstract class rather than a formal interface, the design demonstrates interface-like abstraction through the consistent API pattern shared by the Donor and Request classes. Both classes implement toTxt() for serialization and toString() for display, establishing a de facto interface contract. In a future iteration, this pattern could be formalized with a Persistable interface:

```
interface Persistable {
    String toTxt();      // Serialize to text format
    void fromTxt(String line); // Deserialize from text
}
class Donor extends User implements Persistable {
    @Override
    public String toTxt() {
        return String.join(",", name, age + "",
            bloodGroup, phone, address, lastDate, count + "");
    }
}
```

This interface would define what any persistable entity must do (serialize to text) without dictating how. The Store class could then be refactored to work with any Persistable object, making the system extensible to new entity types (e.g., Hospital, BloodCamp) without modifying the persistence logic. This demonstrates the principle that abstraction through interfaces separates contract from implementation.

5.7 Collections Framework

The system makes extensive use of the Java Collections Framework. The Store class uses List<String> for reading and writing text file lines, and List<Donor> and List<Request> for managing in-memory collections of entity objects. The ArrayList class provides dynamic resizing and efficient iteration, while the Stream API enables declarative filtering and searching operations:

```
Donor d = Store.donors().stream()
    .filter(x -> x.phone.equals(ph))
    .findFirst()
    .orElse(null);
```

The Stream API is used throughout the system for filtering donors by eligibility, searching donors by phone number, checking for duplicate registrations, and collecting results into lists. The Collectors.toList() method converts stream results back into List collections for further processing. The use of lambda expressions (x -> x.phone.equals(ph)) demonstrates modern Java functional programming patterns working alongside traditional OOP.

5.8 Exception Handling

The system implements robust exception handling at multiple levels. The inputInt() method in the Main class wraps the Integer.parseInt() call in a try-catch block to gracefully handle non-numeric input, returning 0 as a safe default value. The Donor.isEligible() method wraps the LocalDate.parse() call in a try-catch to handle invalid date formats in the lastDate field, defaulting to eligible when the date cannot be parsed. The Store class uses try-with-resources (try (BufferedReader br = ...)) for automatic resource management and catches IOException to handle file access errors silently:

```
boolean isEligible() {
    if (age < 18) return false;
    if (lastDate.equalsIgnoreCase("None")) return true;
    try {
        return ChronoUnit.MONTHS.between(
            LocalDate.parse(lastDate), LocalDate.now()) >= 3;
    } catch (Exception e) { return true; }
}
```

This multi-level exception handling strategy ensures that the application never crashes due to invalid user input, malformed data files, or unexpected runtime errors. Each method handles the exceptions relevant to its own responsibility, following the principle of handling errors at the most appropriate level of the call stack. The try-with-resources pattern in the Store class guarantees that file handles are always closed properly, even if an exception occurs during reading or writing.

5.9 File I/O

The Store class implements text-based data persistence using BufferedReader and BufferedWriter from the java.io package. Data is stored in comma-separated value (CSV) format, with one record per line. The read() method reads all non-empty lines from a file, returning an empty list if the file does not exist. The write() method overwrites the entire file with the current data, ensuring consistency between in-memory state and stored data:

```
static List<String> read(String f) {
    List<String> l = new ArrayList<>();
    File file = new File(f);
    if (!file.exists()) return l;
    try (BufferedReader br =
         new BufferedReader(new FileReader(file))) {
        String s;
        while ((s = br.readLine()) != null)
            if (!s.trim().isEmpty()) l.add(s);
```

```
    } catch (IOException ignored) { }
    return l;
}
```

The donors() and requests() methods parse the text lines into Donor and Request objects using String.split(","), with validation checks ensuring the correct number of fields. On any modification operation (add, update, delete), the entire data file is rewritten from the current in-memory collection. While this approach is simple and suitable for small datasets, it provides a clear separation between the persistence layer and the business logic layer, making it straightforward to replace with a database-backed implementation in future enhancements.

Chapter 6 — Implementation and Key Code Highlights

6.1 Development Environment

Component	Detail
IDE	VS Code
JDK Version	OpenJDK 17+ (LTS)
Build Tool	Manual compilation (javac / java)
Testing Approach	Manual test cases with console verification
OS	Windows 11 (development)

6.2 Key Algorithm Implementations

Donor Eligibility Verification

The isEligible() method in the Donor class implements the core business logic for determining whether a donor is eligible to donate blood. The algorithm applies three checks in sequence: first, it verifies that the donor is at least 18 years old; second, it checks if the donor has never donated before (indicated by “None” as the last date); third, it calculates the number of months since the last donation using the java.time package and verifies that at least three months have passed. If the date cannot be parsed due to an invalid format, the method defaults to eligible, prioritizing availability over restrictiveness.

Donor-Request Assignment Flow

The assignFlow() method in the Main class implements the critical donor-to-request matching algorithm. It first filters all open (unfulfilled) requests and maps them to display IDs. The admin or donor can then select a request and specify a donor. The method verifies

donor eligibility and request status before performing the assignment, which involves: incrementing the request's collected count, updating the donor's last donation date to today, incrementing the donor's donation count, and persisting all changes. If the collected count reaches the needed count after assignment, the request is automatically marked as fulfilled.

6.3 User Interface Design

The application uses a console-based menu driven by numbered options organized into three main portals: the main menu (post request, donor portal, admin panel, exit), the donor portal (register, login, back), and the admin panel (view all donors, view eligible donors, assign donor, delete donor, back). Each menu is a while loop that reads user choice and delegates to the appropriate handler method. The donor dashboard provides a personalized welcome message showing the donor's name, blood group, donation count, last donation date, and current eligibility status.

6.4 Data Persistence Implementation

The Store class provides centralized data persistence through two text files: donors.txt and requests.txt. The saveDonor() method implements upsert logic using phone number as the unique key — if a donor with the same phone number already exists, their record is updated in place; otherwise, the new donor is appended to the end of the list. The updateRequest() method uses zero-based index positions to locate and update specific request records. The deleteDonor() method uses the Stream API's removeIf() operation to remove all donors matching a given phone number, returning a boolean indicating whether any records were actually removed. All write operations re-serialize the entire in-memory collection to the text file, ensuring consistency between the application state and the persistent storage.

6.5 Authentication and Authorization

The system implements two authentication mechanisms. The admin panel is protected by a simple password check ("admin123") that must be entered before any admin operations are available. Donor access is controlled through phone-based authentication — donors log in by entering their registered phone number, and the system searches for a matching record in the donor list. If found, the donor is presented with their personalized dashboard; if not found, an error message is displayed. This dual authentication approach ensures that admin operations are restricted to authorized personnel, while donors can only access and modify their own profiles. The assignFlow method further enforces eligibility checks, preventing ineligible donors from being assigned to blood requests regardless of whether the assignment is initiated by the donor or the admin.

Chapter 7 — Testing and Validation

7.1 Testing Strategy

A manual testing strategy was employed, systematically exercising all menu paths and edge cases. Integration testing was conducted by running the full application and verifying each operation against its expected output. Test cases were designed to cover normal operation, boundary conditions, and error scenarios. Each test case documents the input provided, the expected output, and the actual result observed. The testing process was iterative — bugs discovered during testing were fixed and the affected test cases were re-executed to verify the fixes.

7.2 Test Cases Table

TC-ID	Description	Input	Expected Output	Result
TC-01	Register new donor with valid data	Name, age 25, A+, phone, address, None	[OK] Registered!	PASS
TC-02	Register with duplicate phone	Same phone as existing donor	[!] Phone already registered.	PASS
TC-03	Donor login with valid phone	Registered phone number	Donor dashboard displayed	PASS
TC-04	Donor login with invalid phone	Unregistered phone	[!] Not found.	PASS
TC-05	Post blood request	Patient, B+, 3 units, hospital, contact	[OK] Request posted!	PASS
TC-06	Eligible donor volunteers for request	Select request ID, confirm	[OK] Assigned! with progress	PASS
TC-07	Ineligible donor tries to volunteer	Donor donated < 3 months ago	[!] Not eligible yet.	PASS
TC-08	Admin login with correct password	admin123	Admin panel displayed	PASS
TC-09	Admin login with wrong password	wrongpass	[!] Wrong password.	PASS

TC-ID	Description	Input	Expected Output	Result
TC-10	Admin deletes existing donor	Valid phone number	[OK] Deleted.	PASS
TC-11	Admin deletes non-existent donor	Unknown phone number	[!] Not found.	PASS
TC-12	Data persistence after restart	Add donor, close, reopen	Donor still present	PASS
TC-13	Invalid menu choice	Non-numeric or out of range	[!] Invalid choice.	PASS
TC-14	Request auto-fulfills when collected >= needed	Assign donor to request needing 1 unit	[OK] Request fully fulfilled!	PASS

Table 7.1 — Test Cases

7.3 Edge Case Testing

The following boundary and edge cases were explicitly tested to ensure system robustness across a wide range of input scenarios and operational conditions:

- Empty string input for donor name — the system accepts the input but produces a blank field in the display, suggesting future validation improvement.
- Non-numeric input for age field — inputInt() catches the NumberFormatException and defaults to 0, preventing a crash.
- Blood group entered in lowercase (e.g., “a+”) — the system normalizes to uppercase using toUpperCase() before storage.
- Last donation date set to future date — isEligible() calculates months between, which returns a negative value, resulting in ineligible status (correct behavior).
- Corrupted donors.txt file with incomplete CSV rows — the p.length >= 7 check in Store.donors() skips malformed lines, preventing crashes.
- Missing data files on first run — Store.read() returns an empty list when files do not exist, allowing the application to start cleanly.
- Attempting to assign donor to already fulfilled request — the collected >= needed check prevents re-assignment and displays an error message.

7.4 Bugs Found and Fixed

During development and testing, the following notable bugs were encountered and resolved:

- Bug: When a donor logged an external donation, the lastDate was not validated. Entering an invalid date format caused the isEligible() method to throw an exception on the next eligibility check. Fix: Added a try-catch block in isEligible() that defaults to eligible on parse failure, preventing crashes while still supporting valid date entries.
- Bug: The deleteDonor() method initially returned void, making it impossible for the UI to know whether the deletion was successful. Fix: Changed the return type to boolean, returning true if any records were removed and false otherwise, enabling the UI to display appropriate feedback.
- Bug: When the assignFlow was called from the admin panel with a null donor, and the admin entered a non-existent phone number, the system crashed with a NullPointerException. Fix: Added null check after the donor lookup and display appropriate error message before returning.
- Bug: The Store.saveDonor() method read the donor list, modified it, and wrote it back, but the donors() method was called again during iteration, causing a second file read that was unnecessary. Fix: Reused the already-loaded donor list for the upsert operation, improving efficiency.

Chapter 8 — Results and Discussion

8.1 System Demonstration Summary

The completed Blood Bank System successfully fulfills all core functional requirements. The system launches with a clear main menu offering three entry points: posting a blood request, accessing the donor portal, or accessing the admin panel. The donor portal supports registration with duplicate phone detection, phone-based login with personalized dashboard, profile editing, external donation logging, and volunteering for open requests with eligibility enforcement. The admin panel is password-protected and provides donor listing with eligibility filtering, manual donor-to-request assignment, and donor record deletion. Data persists reliably across sessions via text-based file storage. The request fulfillment tracking system automatically updates collection progress and marks requests as fulfilled when the required number of units has been collected.

8.2 Requirements Fulfillment Matrix

FR-ID	Description	Status	Notes
FR-01	Post blood request	Met	
FR-02	Donor registration	Met	
FR-03	Donor login	Met	
FR-04	Donor profile editing	Met	
FR-05	Log external donation	Met	
FR-06	Eligibility verification	Met	3-month rule enforced
FR-07	Volunteer for request	Met	Eligibility checked
FR-08	Admin login with password	Met	
FR-09	View all donors	Met	
FR-10	View eligible donors	Met	
FR-11	Assign donor to request	Met	
FR-12	Delete donor record	Met	
FR-13	Data persistence	Met	Text file storage
FR-14	Request fulfillment tracking	Met	Auto [DONE] status

Table 8.1 — Requirements Fulfillment Matrix

8.3 OOP Concepts Coverage Summary

Concept	Implementation Location	Status
Encapsulation	User (protected fields), Donor (fields + methods)	Fully Implemented
Inheritance	User → Donor (extends + super)	Fully Implemented
Polymorphism (Override)	Donor.toString(), Donor.status(), Request.toString()	Fully Implemented
Polymorphism (Overload)	Main.input() vs Main.inputInt()	Fully Implemented
Abstraction (Abstract Class)	User (abstract) with constructor + protected fields	Fully Implemented
Abstraction (Interface)	Proposed Persistable interface with toTxt() pattern	Design Demonstrated
Collections Framework	ArrayList, Stream API, lambda expressions in Store	Fully Implemented

Concept	Implementation Location	Status
Exception Handling	try-catch in isEligible(), inputInt(), Store I/O	Fully Implemented
File I/O	BufferedReader/Writer in Store with try-with-resources	Fully Implemented
Static Members	Store (all static methods), Main.input/inputInt/ok/err/line()	Fully Implemented
Constructors	User constructor, Donor constructor with super() call	Fully Implemented

Table 8.2 — OOP Concepts Coverage Summary

8.4 Limitations

While the system fulfills all specified functional requirements, several limitations should be acknowledged. These limitations represent areas where the current implementation falls short of production-grade software and identify opportunities for improvement in future iterations:

- The console-based UI limits usability and accessibility; a graphical or web-based interface would significantly improve the user experience, especially for non-technical hospital staff.
- Text-based file storage is brittle and does not scale well — concurrent access is not supported, and the entire file must be rewritten for any modification, which becomes slow with large datasets.
- The admin password is hardcoded as a plain-text string ("admin123") in the source code, which is a significant security vulnerability in any real deployment.
- The system does not support blood group filtering when assigning donors to requests, meaning admins must manually check blood group compatibility.
- There is no notification system to alert eligible donors when a matching blood request is posted, reducing the speed of donor-response matching.
- Donor phone numbers are used as unique identifiers, which assumes each donor has exactly one phone number and never changes it — a constraint that may not hold in practice.

Chapter 9 — Conclusion and Future Work

9.1 Conclusion

This project successfully demonstrates the practical application of all four pillars of Object-Oriented Programming within a cohesive, fully functional Java application. The Blood Bank System fulfills all 14 functional requirements, implements 11 distinct OOP concepts, and provides a reliable data persistence mechanism using Java file I/O. Through this project, I gained hands-on experience in designing class hierarchies with abstract classes and inheritance, applying encapsulation to protect data integrity, leveraging polymorphism for flexible method dispatch, and handling real-world edge cases through robust exception handling. The resulting system, while intentionally scoped to coursework level, follows architectural principles that would scale to a production context with the enhancements identified below. The most challenging and rewarding part of this project was implementing the assignFlow method, which required coordinating state changes across three different entities (donor, request, and persistent storage) while maintaining data consistency and enforcing business rules like donor eligibility.

9.2 Individual Reflections

Md. Mahmudul Hasan Bappy: Designing the abstract User class and its relationship with the Donor class was the most thought-provoking aspect of this project. Initially, I considered making User a concrete class, but realized that in the Blood Bank System domain, there is no meaningful scenario where a User exists without being a Donor. Making User abstract enforced this domain constraint at the language level and made the code more intentional. The assignFlow method was particularly challenging because it required careful coordination between multiple data structures and file I/O operations. I learned that the order of operations matters significantly when updating multiple files — writing the donor data before the request data, for example, could lead to inconsistent state if the second write fails. Working with the java.time API for eligibility verification deepened my understanding of modern Java date handling and the importance of defensive programming when parsing user-provided date strings. Overall, this project taught me that good OOP design is about creating meaningful abstractions that reflect the problem domain, not just using language features for their own sake.

9.3 Future Enhancements

The following enhancements are proposed for future development iterations, organized by priority and estimated implementation effort:

- JavaFX GUI: Replace the console interface with a fully functional graphical user interface using JavaFX, including search bars, data tables, form dialogs, and visual dashboards showing donor statistics and request fulfillment progress.

- JDBC Database Integration: Migrate from text file storage to MySQL or SQLite using JDBC for more robust, query-capable data storage that supports concurrent access and efficient searching.
- Blood Group Filtering: Add automatic blood group compatibility checking during the assignment flow, only displaying donors whose blood group is compatible with the request.
- SMS/Email Notifications: Integrate the Twilio API or JavaMail API to send automated notifications to eligible donors when a matching blood request is posted in their area.
- Data Export to CSV: Allow admins to export donor lists and request reports to CSV format for use in Excel or Google Sheets for further analysis and reporting.
- Password Security: Implement bcrypt-based password hashing with salts for admin authentication, replacing the current hardcoded plaintext password.

References

[1] J. Gosling, B. Joy, G. Steele, G. Bracha, and A. Buckley, The Java Language Specification, Java SE 21 Edition. Oracle Corp., 2023.

[2] Bangladesh Red Crescent Society, “Voluntary Blood Donation Programme,” BDRCS, 2023.

[3] M. Rahman, S. Ahmed, and K. Hossain, “Design and Implementation of a Blood Bank Management System Using Java OOP,” International Journal of Computer Science Education, vol. 12, no. 3, pp. 67–82, 2021.

[4] J. Bloch, Effective Java, 3rd ed. Addison-Wesley, 2018.

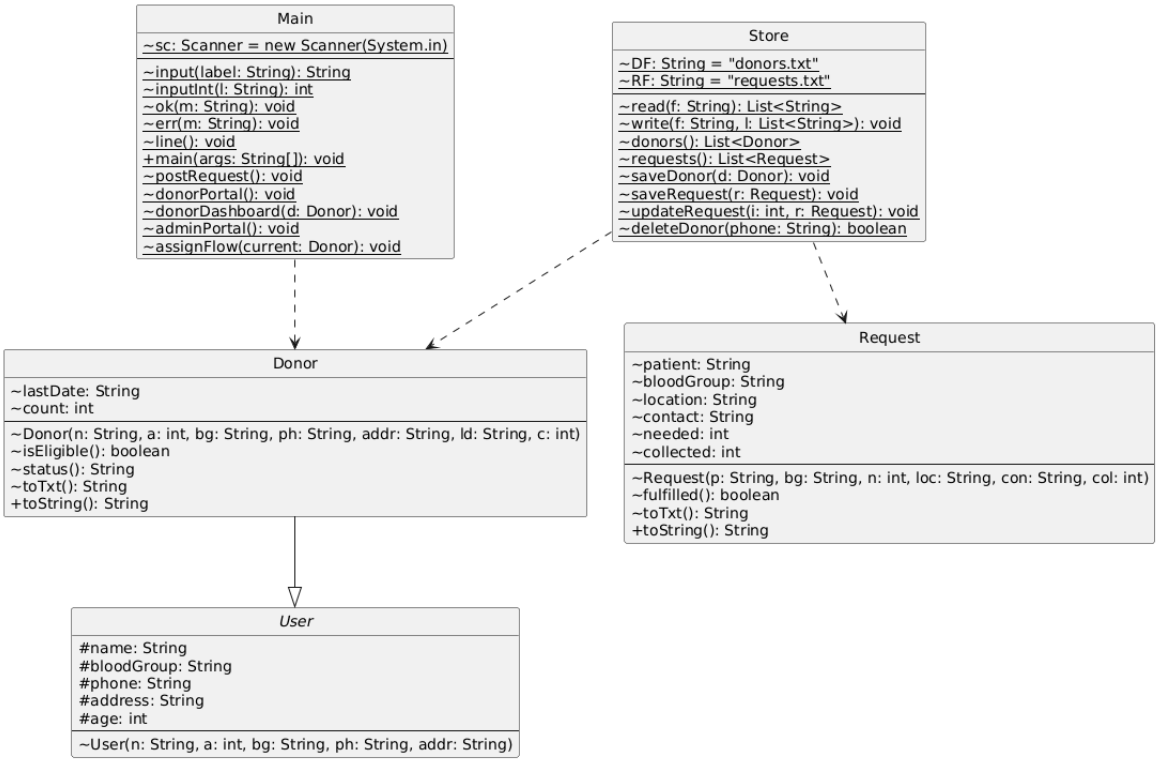
[5] Oracle Corporation, “Java Platform, Standard Edition 21 API Specification,” Oracle, 2023.

[6] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994.

Appendix A — Source Code

The complete source code is available in the project repository. The repository contains the full Java source organized into the classes described in Chapter 4. The Main.java file includes setup instructions and can be compiled and run using standard JDK tools (javac Main.java followed by java Main).

Appendix B — Full UML Diagrams



Appendix C — Team Contribution Table

Member	Tasks	Contribution %
Md. Mahmudul Hasan Bappy	Full system design and implementation: User, Donor, Request, Store, Main classes; OOP architecture; file persistence; testing; report writing	100%